
(두더지게임) 개발보고서

이름 : 이나연

1

개발 목표와 특징

□ 개발하고자 하는 제안시스템의 특징을 반영한 개발목표를 기술

두더지게임은 사용자의 반응 속도 향상과 몰입도 높은 게임 경험을 제공하기 위해 설계된 아두이노 기반 인터랙티브 시스템이다.

LED, 부저, LCD 등을 활용한 피드백과 직관적인 버튼 조작을 통해 누구나 쉽게 즐길 수 있도록 하며, 시간이 지날수록 두더지의 출현 속도를 높여 난이도를 점진적으로 증가시킨다.

반응 속도와 집중력을 테스트할 수 있는 요소를 포함해 단순한 게임을 넘어 학습 효과까지 기대할 수 있다.

2

개발 필요성

□ 왜 본 시스템 개발이 필요한지 기술적 환경 및 수요 배경을 중심으로 기술

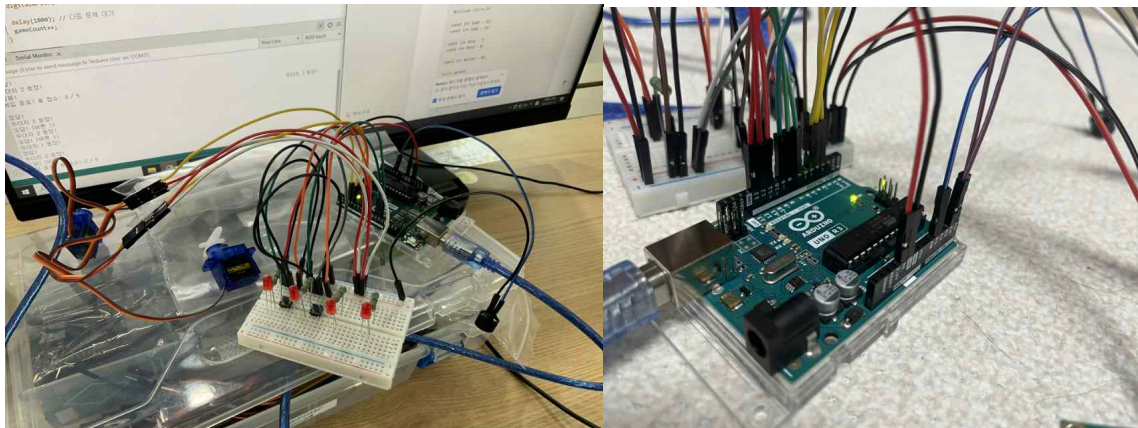
단순한 출력 중심의 프로젝트보다 사용자와 상호작용하는 인터랙티브 콘텐츠에 대한 수요가 증가하고,

두더지게임은 반응형 동작과 게임 요소를 결합하여 높은 몰입감과 실습 효과를 동시에 제공한다.

또한 서보모터, 버튼, 부저, LCD 등 다양한 전자부품을 통합적으로 제어할 수 있어, 하나의 프로젝트로 여러 개념을 익힐 수 있는 장점이 있다. 그 결과 아두이노를 처음 접하는 학습자도 흥미를 느끼며 쉽게 전자제어 개념을 익힐 수 있다.

□ H/W를 어떻게 구현했는지를 그림, 사진 등으로 설명, 사용 센서, 부품들을 나열

부품명	수량	구현
아두이노UNO	1개	메인보드
브레드보드	1개	임시 회로 구성
점퍼선	여러개	연결
SG90 서보모터	4개	두더지의 움직임 제어
택트 스위치	4개	두더지의 출몰 여부를 감지하고 반응을 입력
수동형 부저	1개	효과음
일반 LED	4개	게임 난이도 3단계에서 사용
330Ω 저항	4개	LED에서 같이 사용
LCD	1개	게임 점수 및 안내 문장 표시



부품 종류	연결된 핀 번호
서보모터	2,3,4,5
LED	10,11,12,13
택트스위치	6,7,8,9
수동형 부저	A1
LCD	SDA: A4, SCL: A5

4

S/W 구현

□ S/W를 어떻게 구현했는지를 설명

1. 게임 시작 전 카운트다운 3초 메시지를 LCD 화면에 출력

2. 두더지게임은 총 3단계로 구성,

단계마다 제한시간동안 올라오는 두더지(서보모터)를 텍스트 스위치로 눌러 점수 획득(정답: 10점 / 오답: -5점)

- 1단계(20초): 3초간격으로 두더지 출몰, 50점 이상 득점시 2단계 진출

- 2단계(20초): 1.5초간격으로 두더지 출몰, 100점 이상 득점시 3단계 진출

- 3단계(20초): 1.5초간격으로 두더지 출몰 & LED로 두더지와 다른 위치에 불이 켜져 방해

5

구현 소감

□ 개발 과정에 있었던 일과 느낌을 소개(개인별)

이번 프로젝트를 진행하면서 처음에는 회로 연결이 가장 어려울 것 같아 막막했지만, LED 연결부터 차근차근 시작하니 생각보다 쉽게 회로를 구성할 수 있었다. 특히 H/W 조립 단계에서는 텍스트 스위치와 LED에 연결한 점퍼선이 자꾸 빠져 불편했는데, 점퍼선 피복을 벗겨 직접 연결하는 방식으로 문제를 해결할 수 있었다.

S/W에서는 LCD 화면을 출력하는 과정에서 오류가 발생하거나 작동이 되지 않는 문제가 있었지만, GPT의 도움을 통해 해결하며 문제 해결 능력을 기를 수 있었다.

6

향후 보완 사항

□ 향후 보완이 필요하다고 생각되는 사항 기술

다음에는 케이스 설계를 사전에 고려하여 부품배치를 계획하고, 고정하기 위한 브래킷 등을 함께 사용하여 두더지가 좀 더 안정적으로 움직일 수 있도록 보완할 필요가 있을 것 같다.

7

참고 문헌

□ 개발 과정에 참고한 도서, 사이트 등을 기술

- S/W: Chat GPT

- H/W: <https://www.youtube.com/watch?v=aTet6a1oSk0>

8

스케치 소스 코드

□ 스케치 코드를 첨부하세요. 가져온 소스는 표시하세요.

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
#include <Servo.h>
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

```
const int NUM_MOLES = 4;
```

```
const int ledPins[NUM_MOLES] = {11, 12, 13, 10};
```

```
const int btnPins[NUM_MOLES] = {7, 8, 9, 6};
```

```
const int servoPins[NUM_MOLES] = {3, 4, 5, 2};
```

```
const int buzzer = A1;
```

```
Servo servos[NUM_MOLES];
```

```
int score = 0;
```

```
void setup() {
```

```
  for (int i = 0; i < NUM_MOLES; i++) {
```

```
    pinMode(ledPins[i], OUTPUT);
```

```
    pinMode(btnPins[i], INPUT_PULLUP);
```

```
    servos[i].attach(servoPins[i]);
```

```
    servos[i].write(0);
```

```
  }
```

```
  pinMode(buzzer, OUTPUT);
```

```
  lcd.init();
```

```
  lcd.backlight();
```

```

lcd.setCursor(0, 0);
lcd.print("Ready to play!");

Serial.begin(9600);
randomSeed(analogRead(0));

for (int i = 3; i > 0; i--) {
    lcd.setCursor(0, 1);
    lcd.print("Start in ");
    lcd.print(i);
    lcd.print("... ");
    delay(1000);
}

lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Game Start!");
delay(1500);
lcd.clear();
}

// GPT도움
void playStage(int duration, int limitTimeMs, bool randomLED) {
    unsigned long startStage = millis();

    while (millis() - startStage < duration) {
        int moleIndex = random(0, NUM_MOLES);
        int ledIndex = moleIndex;

        if (randomLED) {
            ledIndex = random(0, NUM_MOLES);
            digitalWrite(ledPins[ledIndex], HIGH);
        }

        servos[moleIndex].write(90);

        lcd.setCursor(0, 0);
        lcd.print("Score: ");
    }
}

```

```

lcd.print(score);
lcd.setCursor(0, 1);
lcd.print("Hit the mole!   ");

Serial.print("두더지 ");
Serial.print(moleIndex + 1);
if (randomLED) {
    Serial.print(" (LED 방해: ");
    Serial.print(ledIndex + 1);
    Serial.print(")");
}
Serial.println();

unsigned long appearTime = millis();
bool answered = false;

while (millis() - appearTime < limitTimeMs && !answered) {
    for (int i = 0; i < NUM_MOLES; i++) {
        if (digitalRead(btnPins[i]) == LOW) {
            if (i == moleIndex) {
                tone(buzzer, 1000, 300);
                score += 10;
                Serial.println("정답! +10점");
            } else {
                tone(buzzer, 500, 200);
                score -= 5;
                Serial.print("오답! 버튼 ");
                Serial.print(i + 1);
                Serial.println(" -5점");
            }
            answered = true;
            break;
        }
    }
}

servos[moleIndex].write(0);

```

```
        if (randomLED) {
            digitalWrite(ledPins[ledIndex], LOW);
        }

        delay(500);
        lcd.clear();
    }
}

void loop() {
    // Stage 1
    lcd.setCursor(0, 0);
    lcd.print("Stage 1");
    delay(1000);
    lcd.clear();
    playStage(20000, 3000, false);

    if (score < 50) {
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("Need 50 points");
        lcd.setCursor(0, 1);
        lcd.print("Try again!");
        Serial.println("스테이지 1 실패. 50점 미만.");
        while (true);
    }

    // Stage 2
    lcd.setCursor(0, 0);
    lcd.print("Stage 2");
    delay(1000);
    lcd.clear();
    playStage(20000, 1500, false);

    if (score < 100) {
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("Need 100 points");
    }
}
```

```
    lcd.setCursor(0, 1);
    lcd.print("Try again!");
    Serial.println("스테이지 3 잠금. 100점 미만.");
    while (true);
}

// Stage 3
lcd.setCursor(0, 0);
lcd.print("Stage 3");
delay(1000);
lcd.clear();
playStage(20000, 1500, true);

lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Game Over!");
lcd.setCursor(0, 1);
lcd.print("Final: ");
lcd.print(score);
Serial.print("게임 종료! 최종 점수: ");
Serial.println(score);

while (true);
}
```